

Names, Bindings, and Scopes

Names

A **name** is a string of characters used to identify some entity in a program. Names in programming languages have the same form: starting with a letter followed by a string consisting of letters, digits, and underscore (_) characters. The term “*identifier*” can be used interchangeably with “name”.

The use of underscore to form names was used in the 1970s-1980s but is now far less popular. In the C-based languages, it has been replaced by *camel notation* or *camel case*, in which all of the words of a multiple-word name except the first are capitalized, as in `myFirstCode`. It is called “*camel*” as words written as names often have embedded uppercase letters, making it look like a camel’s hump.

Here are some naming conventions for various programming languages:

- **Java** and **C#** have no length limit, with all characters being significant
- **C++** does not specify length limits, but implementors sometimes do
- All variable names in **PHP** must begin with a dollar (\$) sign
- The special character at the beginning of a variable’s name in **Perl**, such as \$, @, or %, specifies its type
- The special characters at the beginning of a variable’s name in **Ruby**, such as @ or @@, specify that the variable is an instance or a class variable, respectively

Additionally, C-based languages have distinct use of uppercase and lowercase letters in names. Names in these languages are case-sensitive. For example, in C++, `kind`, `KIND`, and `Kind` are distinct. There is no connection between them.

Here is an outline of some of the common naming styles in Python code and when they should be used.

Type	Naming Convention	Examples
Function	Use lowercase words. Separate words by underscores to improve readability. This is called a snake case .	<code>function</code> , <code>pyt_function</code>
Variable	Use a lowercase single letter, word, or words. Separate words with underscores (_) to improve readability.	<code>z</code> , <code>var</code> , <code>pyt_variable</code>
Class	Start each word with a capital letter. Do not separate words with underscores. This is called a Pascal case *	<code>Model</code> , <code>PythonClass</code>
Method	Use lowercase words. Separate words with underscores to improve readability (snake case).	<code>class_method</code> , <code>method</code>
Constant	Use an uppercase single letter, word, or words. Separate words with underscores to improve readability.	<code>CONSTANT</code> , <code>PYTH_CONSTANT</code> , <code>PYTHON_LONG_CONSTANT</code>
Module	Use a short, lowercase word or words. Separate words with underscores to improve readability.	<code>module.py</code> , <code>python_module.py</code>
Package	Use a short, lowercase word or words. Do not separate words with underscores.	<code>package</code> , <code>pythonpackage</code>

*Pascal case is a naming convention wherein each word starts with a capital letter.

Special Words

These are used to make programs more readable by naming actions to be performed. They separate the syntactic parts of statements and programs. In most languages, these are known as *reserved words* that cannot be redefined by programmers. But in some cases, such as Fortran, they are only **keywords** wherein they can be redefined.

A **reserved word** is a special word that cannot be used as a name in a programming language. This becomes difficult and a problem for programmers if the language includes many reserved words. For example, COBOL has 300 reserved words wherein some of the most commonly chosen names are in the list of reserved words such as LENGTH, BOTTOM, DESTINATION, and COUNT. Most times, a programming environment would assign specific text colors to reserved words once used.

Here are the identifiers used as reserved words in Python.

- False
- None
- True
- and
- as
- assert
- async
- await
- break
- class
- continue
- def
- del
- elif
- else
- except
- finally
- for
- from
- global
- if
- import
- in
- is
- lambda
- nonlocal
- not
- or
- pass
- raise
- return
- try
- while
- with
- yield

Variables

A variable in programming is an abstraction of a computer memory cell or collection of cells. It can be characterized as a sextuple of attributes including name, address, type, value, lifetime, and scope.

- **Variable Names** – the most common names in programs. These are used to identify and reference values that can change. They can be made up of letters, numbers, and underscores and should be descriptive and meaningful.
- **Variable Address** – the address of a variable is the unique machine memory address it is associated with. It acts as a numerical identifier that allows the program to access and manipulate the data held by the variable.
- **Variable Type** – determines the range of values the variable can store and the set of operations defined for values of the type. Examples include int, float, and Boolean.
- **Variable Value** – the contents of the memory cell or cells associated with the variable. A variable's value can also be called its **r-value**, as it is required when the variable name appears on the right side of an assignment statement.
- **Variable Lifetime** – the amount of time it exists and retains its value in memory, depending on how and where it is declared.
- **Variable Scope** – the part of the program where a variable can be accessed. It is determined by where the variable is declared in the code.

Bindings

Binding is an association between an attribute and an entity, such as between a variable and its type or value, or between an operation and a symbol. **Binding time** is the time a binding takes place, for instance, at language design time, language implementation time, compile time, load time, link time, or run time.

For example, the asterisk (*) symbol is often bound to the multiplication operation at language design time. A data type, such as int, is bound to a range of possible values at language implementation time. A variable in a Java program is bound to a particular data type at compile time.

Using this assignment statement:

```
count = count - 3
```

Here are some of the bindings and their binding times for the parts of this assignment:

- The type of `count` is bound at compile time
- The set of possible values of `count` is bound at compiler design time
- The meaning of the operator symbol `-` is bound at compile time, when the types of its operands have been determined
- The internal representation of the literal `3` is bound at compiler design time
- The value of the `count` is bound at execution time with this statement

A clear understanding of the binding times for the attributes of program entities is necessary to understand the semantics of a programming language. For instance, to determine the current value of a variable, it can be essential to know when the variable was bound to storage and with which statement or statements.

Type Bindings

A variable must be bound to a data type before being referenced in a program. Aside from the binding time, it is also important to know how the type is specified. Types can be defined statically by some form of explicit or implicit declaration.

An **explicit declaration** is a statement in a program that lists variable names and specifies their particular type. It directly specifies the data type of the variable when declaring it.

For example, in C:

```
int number; // Explicitly declares 'number' as an integer [1, 2, 5]
float decimal; // Explicitly declares 'decimal' as a floating-point number [1.3, 2.5]
char letter; // Explicitly declares 'letter' as a character [A, B, C]
```

An **implicit declaration** is a means of associating variables with types through default conventions rather than declaration statements. It automatically assigns the variable's data type based on its usage within the code without an explicit declaration statement. Both declarations create **static bindings** to types.

For example, in a hypothetical language with implicit declaration:

```
// No explicit declaration for "result"
result = 5 + 2.3
print(result) // This would print "7.3" implying "result" was implicitly treated as a
floating-point number
```

In Python, variables are dynamically typed, so it is unnecessary to declare their types explicitly. However, the concept of **explicit declaration** can still be applied by binding a variable to a value and showing that the variable can later change its type.

For example, a program to demonstrate explicit declaration and binding:

```
# Explicit variable declaration and binding

# Binding an integer to variable 'x'
x = 10
print(f"x is bound to {x} and its type is {type(x)}")

# Now, binding a string to the same variable 'x'
x = "Hello, world!"
print(f"x is now bound to '{x}' and its type is {type(x)}")

# Binding a float to 'x'
x = 3.14
print(f"x is now bound to {x} and its type is {type(x)}")

# Binding a List to 'x'
x = [1, 2, 3]
print(f"x is now bound to {x} and its type is {type(x)}")
```

While Python does not require explicit type declaration, the **binding** occurs when a value is assigned to a variable. Each time a value is assigned to x, Python "binds" that value to the variable.

The type of the variable x can change dynamically, depending on the value that is bound to it. For example, x is first bound to an integer, then a string, then a float, and finally a list.

The program uses the type() function to print the type of x after each binding.

The output of this program is:

```
x is bound to 10 and its type is <class 'int'>
x is now bound to 'Hello, world!' and its type is <class 'str'>
x is now bound to 3.14 and its type is <class 'float'>
x is now bound to [1, 2, 3] and its type is <class 'list'>
```

This shows the explicit binding of values to the variable x and how its type changes dynamically based on the value it holds.

Python is dynamically typed, meaning it is unnecessary to declare the type of a variable when first assigning it. The type can change as the variable is bound to different values (integer, string, float, list, etc.).

Dynamic type binding happens when the variable type is not specified by a declaration statement, nor can be determined by the spelling of its name. The variable is bound to a type when assigned a value in an assignment stated.

For implicit declaration in Python, it is demonstrated by simply assigning a value to a variable without specifying its type. The variable is automatically assigned a type based on the value it holds.

For example, a Python program demonstrating **implicit declaration** and binding:

```
# Implicit variable declaration and binding
# The variable 'y' is implicitly declared and bound to an integer
y = 42
print(f"y is implicitly bound to {y} and its type is {type(y)}")

# The same variable 'y' is now implicitly bound to a string
y = "Python programming"
print(f"y is now implicitly bound to '{y}' and its type is {type(y)}")

# Now, 'y' is implicitly bound to a float
y = 3.14159
print(f"y is now implicitly bound to {y} and its type is {type(y)}")

# Finally, 'y' is implicitly bound to a list
y = [1, 2, 3, 4]
print(f"y is now implicitly bound to {y} and its type is {type(y)}")
```

In this example, the type of variable `y` is not explicitly declared. Instead, values are assigned to `y`, and Python implicitly determines its type based on the assigned value. Python automatically **binds** the value to the variable and figures out the type behind the scenes.

The output of this program is:

```
y is implicitly bound to 42 and its type is <class 'int'>
y is now implicitly bound to 'Python programming' and its type is <class 'str'>
y is now implicitly bound to 3.14159 and its type is <class 'float'>
y is now implicitly bound to [1, 2, 3, 4] and its type is <class 'list'>
```

Scopes

The **scope** of a variable is the range of statements wherein the variable is visible. A variable is visible in a statement if it can be referenced or assigned in that statement.

Furthermore, scope refers to the region of a program where a particular variable or function is accessible and valid. It determines the **lifetime** and **visibility** of a variable, meaning where it can be used and modified within the code.

Types of Scope:

- **Global Scope:** Variables declared in the global scope are accessible from anywhere in the program, including inside functions or methods. They are usually defined outside of any function or block of code.

Example in Python:

```
x = 10 # Global variable
def my_function():
    print(x) # x is accessible here because it is in global scope
my_function() # This will print 10
```

- **Local Scope:** Variables declared inside a function or block of code are **local variables** and can only be accessed within that specific function or block. They are **not visible** outside their defining scope.

Example in Python:

```
def my_function():  
    y = 5 # Local variable  
    print(y)  
  
my_function() # This will print 5  
print(y) # This will raise an error because 'y' is not accessible outside the function
```

- **Enclosing Scope:** This refers to the scope of a function that **encloses** another function. If there is a nested function, the outer function's variables are available to the inner function, but not the other way around.

Example in Python:

```
def outer_function():  
    z = 15 # Enclosing variable  
    def inner_function():  
        print(z) # Accessing the enclosing scope variable  
    inner_function() # This will print 15  
outer_function()
```

- **Built-in Scope:** The built-in scope contains all the functions and variables that are built into Python, like `print()`, `len()`, and `int()`. These are always available to use.

Example in Python:

```
print(len([1, 2, 3])) # Using built-in function `len`
```

Scope Rules

Here are some scope rules in Python and many other languages.

- **LEGB Rule:** Python follows the LEGB rule when looking up variables, which stands for:
 - Local (innermost scope)
 - Enclosing (any enclosing functions, like in closures)
 - Global (the module-level scope)
 - Built-in (the scope that contains Python's built-in objects and functions)
- **Shadowing:** If a variable declared in a local scope has the same name as a variable in an outer (enclosing or global) scope, it **shadows** the outer variable, meaning the local variable takes precedence. For example,

```
x = 100 # Global variable  
  
def my_function():  
    x = 50 # Local variable, shadows the global variable  
    print(x)  
  
my_function() # Prints 50, because the local x shadows the global x  
print(x) # Prints 100, the global x remains unchanged
```


- **global Keyword:** In Python, to modify a global variable inside a function, it must be declared with a global keyword to avoid creating a local variable with the same name.

For example:

```
x = 10

def modify_global():
    global x # Declare that we are modifying the global variable 'x'
    x = 20

modify_global()
print(x) # Prints 20
```

References:

Sebesta, R. (2019). *Concepts of programming languages*. Pearson.

Finer, J. (2025). *How to write beautiful Python code with PEP 8*. [Web Article]. Retrieved on February 18, 2025, from <https://realpython.com/python-pep8/>